Connect Through Signs

Sequence Modeling

Overview

This document presents the sequence modeling for the Connect Through Signs system. It contains three UML sequence diagrams and three supporting sections that together describe how the system components interact and how errors are handled.

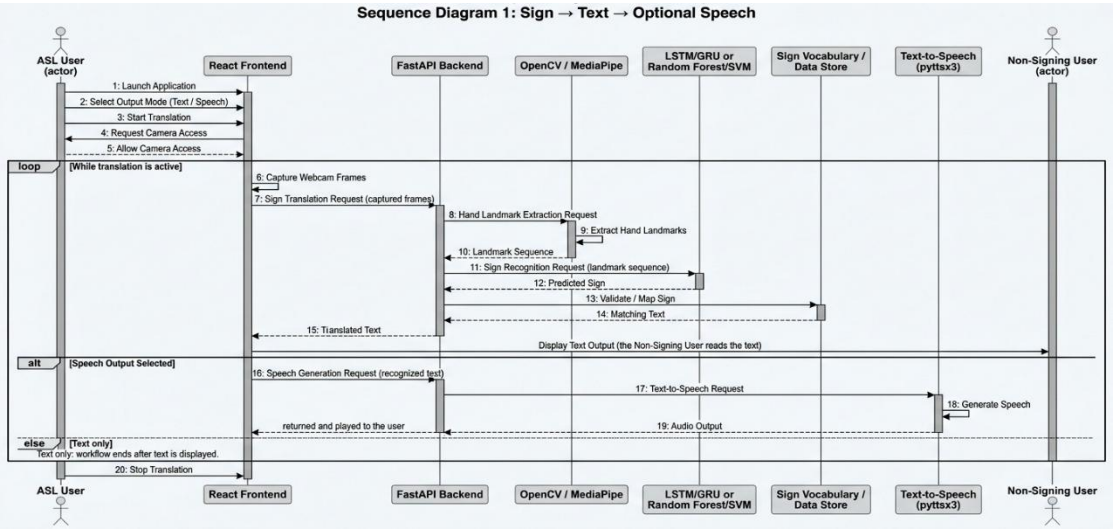
No.	Section	Type
1	Sequence Diagram 1 – Sign → Text → Optional Speech	UML sequence diagram
2	Sequence Diagram 2 – Text/Speech → Sign → 3D Avatar	UML sequence diagram
3	Sequence Diagram 3 – Exception Handling & Fallback	UML sequence diagram
4	Message Flow Mapping	Supporting table
5	Exception Flow Detailing	Supporting table
6	Consistency Verification	Use case ↔ sequence diagram mapping

1. Sequence Diagram 1: Sign → Text → Optional Speech

1.1 Purpose

This diagram models the main functional workflow of the system: an ASL user signs in front of the camera, the system recognizes the sign, translates it into text, displays the text for the Non-Signing User to read, and optionally converts the text into speech.

Workflow: ASL User → Sign Capture → Sign Recognition → Text Translation → Text Output → Optional Speech.

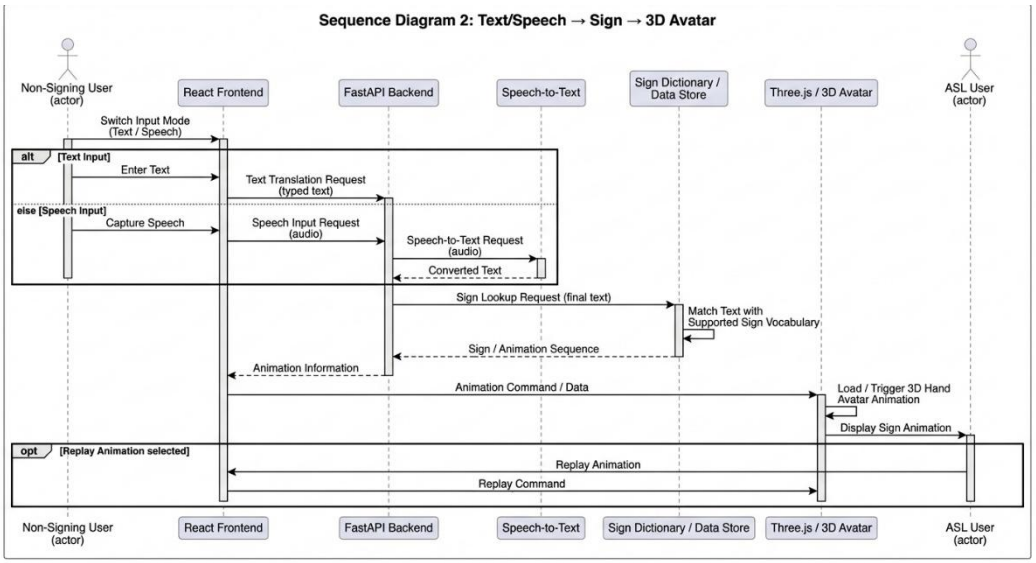


2. Sequence Diagram 2: Text/Speech → Sign → 3D Avatar

2.1 Purpose

This diagram models the reverse communication workflow: a non-signing user provides typed or spoken input, the system converts it to text where necessary, finds the matching sign in the sign dictionary, and displays the sign as an animated 3D hand avatar for the ASL User to watch.

Workflow: Non-Signing User → Text/Speech Input → Text Processing → Sign Translation → Sign Animation.



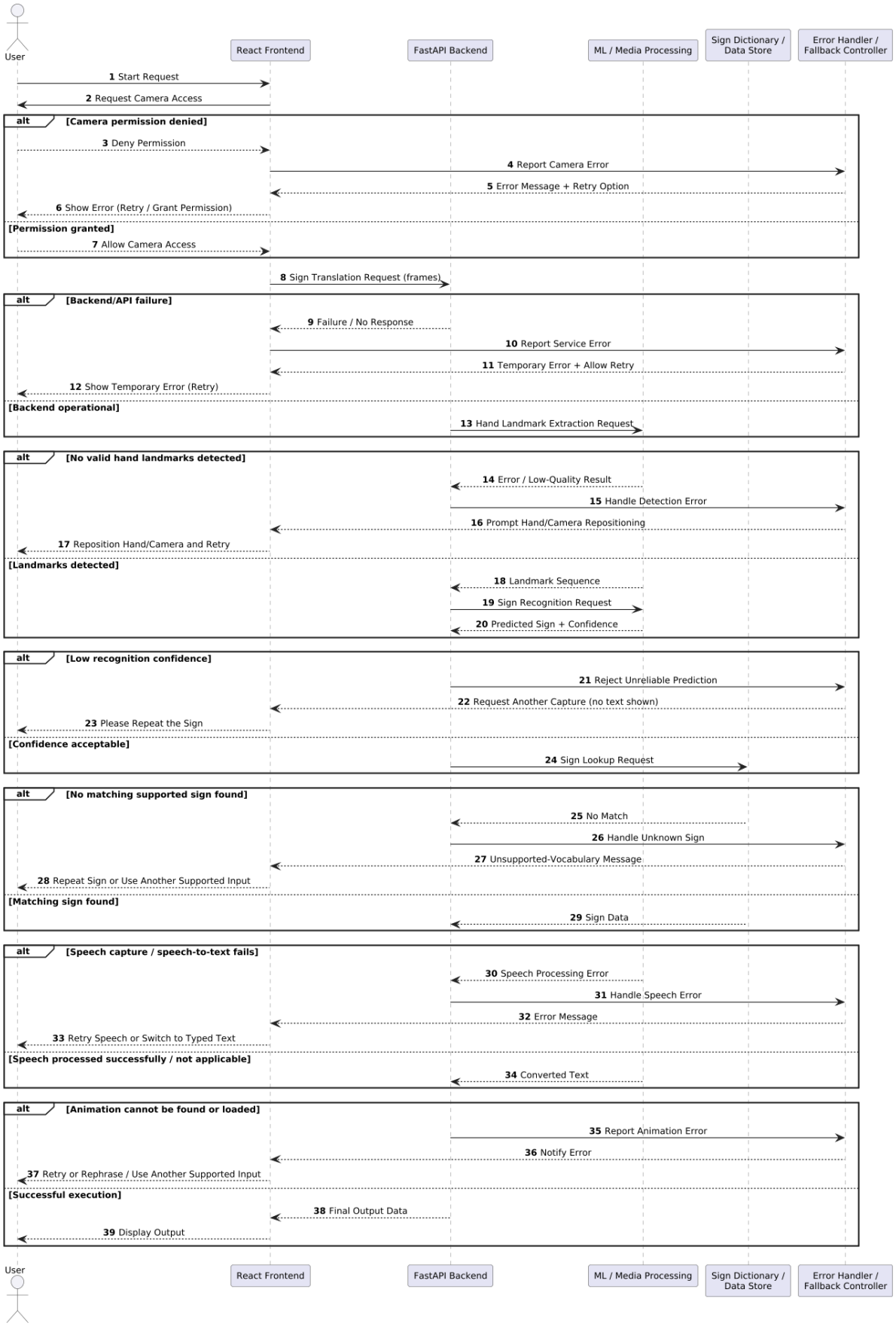
3. Sequence Diagram 3: Exception Handling & Fallback

3.1 Purpose

This diagram addresses the requirement to formulate robust error handling logic and fallback execution paths. It represents the Handle Errors use case, which extends the other use cases in the Use Case Diagram. It is drawn as one common workflow with alternative (alt) failure branches, not as seven separate diagrams.

Sequence Diagram 3: Exception Handling and Fallback Execution Paths

Use Case: Handle Errors



3.2 Participants (left to right)

1. User
2. React Frontend
3. FastAPI Backend
4. ML / Media Processing
5. Sign Dictionary / Data Store
6. Error Handler / Fallback Controller

4. Message Flow Mapping

This section explains how the Frontend, Backend and Data Store communicate. Exact API endpoint names and a specific database system are not defined in the project documentation, so generic message names are used. Each row shows the request message sent by the sender and the response returned.

4.1 Workflow A: Sign → Text → Speech

Step	Sender	Receiver	Message / API Call (Request)	Data Transferred	Response
A1	ASL User	React Frontend	Launch Application	User action	Application interface is loaded
A2	ASL User	React Frontend	Select Output Mode (Text / Speech)	Output mode selection	Selected mode is applied
A3	ASL User	React Frontend	Start Translation Request	User command	Camera access prompt is shown
A4	React Frontend	ASL User	Camera Access Request	Permission request	Camera access granted (or denied – see Section 5)
A5	React Frontend	FastAPI Backend	Sign Translation Request	Captured webcam frames/input	Request received; processing begins
A6	FastAPI Backend	OpenCV / MediaPipe	Hand Landmark Extraction Request	Captured frames/input	Hand landmark sequence
A7	FastAPI Backend	LSTM/GRU or Random Forest/SVM	Sign Recognition Request	Landmark sequence	Predicted sign (with confidence)
A8	FastAPI Backend	Sign Vocabulary / Data Store	Vocabulary Validation/Mapping Request	Predicted sign	Validated sign and corresponding text
A9	FastAPI Backend	React Frontend	Sign Translation Response	Translated text	Frontend displays the text for the Non-Signing User to read
A10	React Frontend	FastAPI Backend	Speech Generation Request (only if speech output selected)	Recognized text	Speech output returned
A11	FastAPI Backend	Text-to-Speech (pyttsx3)	Text-to-Speech Request	Recognized text	Generated speech/audio
A12	FastAPI Backend	React Frontend	Speech Generation Response	Audio output	Audio is played to the user
A13	ASL User	React Frontend	Stop Translation	User command	Capture and recognition loop ends

4.2 Workflow B: Text/Speech → Sign → Avatar

Step	Sender	Receiver	Message / API Call (Request)	Data Transferred	Response
B1	Non-Signing User	React Frontend	Switch Input Mode	Mode selection (text or speech)	Selected input option is displayed
B2	Non-Signing User	React Frontend	Enter Text / Capture Speech	Typed text or spoken audio	Input accepted by Frontend
B3	React Frontend	FastAPI Backend	Text Translation Request (text mode)	Typed text	Request received; processing begins
B4	React Frontend	FastAPI Backend	Speech Input Request (speech mode)	Captured audio	Request received; processing begins
B5	FastAPI Backend	Speech-to-Text	Speech-to-Text Request	Audio	Converted text
B6	FastAPI Backend	Sign Dictionary / Data Store	Sign Lookup Request	Final text	Corresponding sign/animation sequence
B7	FastAPI Backend	React Frontend	Sign Translation Response	Animation information	Frontend receives the animation data
B8	React Frontend	Three.js / 3D Avatar	Animation Command	Animation command/data	Avatar loads/triggers the animation
B9	Three.js / 3D Avatar	ASL User	Display Sign Animation	Rendered 3D hand animation	Animated sign is shown to the ASL User
B10	ASL User	React Frontend	Replay Animation Request	Replay command	Animation is replayed by Three.js

4.3 Summary of Communication Paths

- Frontend → Backend: translation requests carrying captured frames, text or audio.
- Backend → ML/Media Processing: landmark extraction and sign recognition requests.
- Backend → Sign Dictionary / Data Store: vocabulary validation and sign lookup requests.
- Backend → Speech Processing: speech-to-text and text-to-speech requests.
- Frontend → Three.js / 3D Avatar: animation commands and replay commands.

5. Exception Flow Detailing

This table explains the logic behind Sequence Diagram 3. The fallback never silently produces an incorrect translation: where recognition is uncertain, the system asks the user to retry instead of presenting an unreliable result as correct.

Error Condition	Detection Point	System Response	Fallback Action	User Action
1. Camera permission denied	React Frontend (camera access request)	Displays an error message explaining that camera access is required	Translation does not start; the system waits for permission	Grant permission or retry

Error Condition	Detection Point	System Response	Fallback Action	User Action
2. No hand detected	OpenCV / MediaPipe (landmark extraction)	Returns an error/low-quality result; Error Handler instructs the Frontend to prompt the user	Discards the frame/sequence; no translation is produced	Reposition hand/camera and retry
3. Low-confidence sign recognition	FastAPI Backend (after ML prediction)	Rejects the prediction as unreliable; Error Handler requests another capture	No text is shown for the uncertain prediction	Repeat the sign
4. Backend/API failure	React Frontend (request fails or times out)	Displays a temporary error message	Allows the request to be retried	Retry the request
5. Unknown/unsupported sign	Sign Dictionary / Data Store (lookup)	Error Handler returns an unsupported-vocabulary message	No incorrect sign or text is substituted	Repeat the sign or use another supported input
6. Speech-to-text failure	Speech-to-Text / FastAPI Backend	Displays an error message	Offers retry of speech input or switching to typed text	Retry speech or type the text
7. Missing/unavailable sign animation	Three.js / 3D Avatar (animation load)	Error Handler informs the Frontend	No wrong animation is played	Retry, or rephrase/use another supported input

6. Consistency Verification

This section verifies the three sequence diagrams against the Revised Use Case Diagram of Connect Through Signs. Actors and relationships are taken directly from that diagram. Status values are: Consistent, Partially Covered, or Not Detailed (no use case is currently Partially Covered). Use cases without sufficient detail in the project proposal are marked as requiring further specification; no behavior has been invented for them.

6.1 Use Case Mapping

Use Case	Actor (per Use Case Diagram)	Sequence Diagram	Corresponding Interaction	Consistency Status	Note
Launch Application	ASL User	SD 1, SD 3	SD 1: step 1 (launch); SD 3: Handle Errors extends this use case	Consistent	
Allow Camera Access	ASL User	SD 1, SD 3	SD 1: camera access request and permission; SD 3: Scenario 1 (denied)	Consistent	
Select Output Mode (Text / Speech)	ASL User	SD 1	SD 1: step 2 (select output mode) and alt [Speech Output Selected]	Consistent	
Start / Stop Translation	ASL User	SD 1	SD 1: step 3 (start), loop [While translation is active], step 20 (stop)	Consistent	
Capture Sign	ASL User	SD 1	Frontend captures webcam frames	Consistent	

Use Case	Actor (per Use Case Diagram)	Sequence Diagram	Corresponding Interaction	Consistency Status	Note
Recognize Sign	ASL User (included by Capture Sign)	SD 1, SD 3	MediaPipe landmark extraction and ML prediction; SD 3: Scenarios 2 and 3	Consistent	
Translate to Text	ASL User (included by Recognize Sign)	SD 1	Backend maps the sign via Sign Vocabulary and returns text	Consistent	
View Text Output	Non-Signing User	SD 1	Frontend displays the text for the Non-Signing User to read	Consistent	Actor aligned with the diagram
Generate Speech	No direct actor (extends Translate to Text)	SD 1	alt [Speech Output Selected]: pyttsx3 generates speech	Consistent	Optional extension matches alt fragment
Switch Input Mode	Non-Signing User	SD 2, SD 3	SD 2: input mode selection (alt Text / Speech); SD 3: Scenario 6 (switch to typed text)	Consistent	
Enter Text	Non-Signing User	SD 2	alt [Text Input]	Consistent	
Capture Speech	Non-Signing User	SD 2, SD 3	alt [Speech Input]; SD 3: Scenario 6	Consistent	
Convert Speech to Text	Non-Signing User	SD 2, SD 3	Backend sends audio to Speech-to-Text and receives text	Consistent	
Translate to Sign	No direct actor (included by Enter Text and Convert Speech to Text)	SD 2, SD 3	Sign Dictionary lookup; SD 3: Scenario 5	Consistent	
Display Sign Animation	ASL User	SD 2, SD 3	Three.js triggers the 3D avatar animation for the ASL User; SD 3: Scenario 7	Consistent	Actor aligned with the diagram
Replay Animation	No direct actor (extends Display Sign Animation)	SD 2	Replay Animation action by the viewer of the animation	Consistent	Optional extension
Handle Errors	No direct actor (extends several use cases)	SD 3	Seven alt failure branches with Error Handler / Fallback Controller	Consistent	
View Conversation History	Both actors (shared)	None	Not modeled in any sequence diagram	Not Detailed	Requires further specification
Manage Settings	Both actors (shared)	None	Not modeled in any sequence diagram	Not Detailed	Requires further specification

6.2 Include / Extend Relationship Check

Relationship in Use Case Diagram	Representation in Sequence Diagrams	Status
Capture Sign «include» Recognize Sign	SD 1: frame capture is always followed by landmark extraction and ML prediction	Consistent

Relationship in Use Case Diagram	Representation in Sequence Diagrams	Status
Recognize Sign «include» Translate to Text	SD 1: predicted sign is mapped through the Sign Vocabulary to text	Consistent
Translate to Text «include» View Text Output	SD 1: translated text is always returned and displayed	Consistent
Generate Speech «extend» Translate to Text	SD 1: optional alt fragment [Speech Output Selected]	Consistent
Enter Text «include» Translate to Sign	SD 2: alt [Text Input] leads to Sign Dictionary lookup	Consistent
Capture Speech «include» Convert Speech to Text	SD 2: alt [Speech Input] sends audio to Speech-to-Text	Consistent
Convert Speech to Text «include» Translate to Sign	SD 2: converted text converges with typed text before the Sign Dictionary lookup	Consistent
Translate to Sign «include» Display Sign Animation	SD 2: matched sign sequence is always sent to Three.js for display	Consistent
Replay Animation «extend» Display Sign Animation	SD 2: optional Replay Animation action after display	Consistent
Handle Errors «extend» (several use cases)	SD 3: alt failure branches for camera, detection, confidence, backend, dictionary, speech and animation errors	Consistent

6.3 Conclusion

Of the 19 use cases verified, 17 are Consistent and 2 are Not Detailed (View Conversation History and Manage Settings). All 10 include/extend relationships checked are reflected in the sequence diagrams. The two Not Detailed use cases require further specification in the project proposal before a sequence can be modeled.

Actor alignment: in the Use Case Diagram the Non-Signing User reads the translated text output, and the ASL User watches the translated sign animation. Sequence Diagrams 1 and 2 and the Message Flow Mapping follow this convention.